

****npm mirrors became phishing hotbeds**** 24 compromised ****npm**** packages redirected $\approx$ 5,000 users to fake Cloudflare-style CAPTCHA pages in a single day. How does a supply-chain slip past our ****Zero-Trust**** gates?

THE SOC PLAYBOOK Our **SOC** monitors every request to public registries. When the **unpkg** CDN spiked with 404s, our **SIEM** flagged the anomaly. Correlating the **IOC** with a known **TTP**—mass-redirect to a phishing landing page—triggered an automated **SOAR** play. It isolated the offending packages and forced a quarantine on the originating **npm** accounts.

****⚠ TECHNICAL RADIANCE**** The attackers published a single HTML file inside each ****npm**** package. The file loads a script that swaps the legitimate ****unpkg**** URL for a clone. It mimics Cloudflare's "Just a moment..." challenge. Because ****unpkg**** mirrors are trusted by default, the malicious page inherits the same ****trust**** level as any legitimate asset. It bypasses naïve content-validation rules.

******  **ZERO-TRUST LESSONS**** *Identity-centric verification* must extend beyond user logins to every artifact that enters the runtime environment.

- ****What most teams assume**** – “If a CDN is public, its content is safe by definition.”
- ****What actually happens**** – “A compromised package can inherit the CDN’s trust, weaponizing the same path we use for legitimate updates.”

Embedding ****IAM**** checks at the package-pull stage verifies publisher